# Differential and Correlation Power Analysis Attacks on HMAC-Whirlpool

Fan Zhang and Zhijie Jerry Shi
Department of Computer Science and Engineering
University of Connecticut, Storrs, CT, USA, 06269
Email: {fan.zhang, zshi}@engr.uconn.edu

***Abstract*—In cryptography, a keyed-Hash Message Authentication Code (HMAC) is a type of message authentication code (MAC) calculated with a cryptographic hash function and a secret key. The security of the HMAC relies on the underlying hash function and the secret key. Whirlpool is a block cipher based hash algorithm that has been in public for about ten years. So far no effective attacks have been found on Whirlpool. As a result, HMAC with Whirlpool, *i.e.*, HMAC-Whirlpool, is supposed to be secure. In this paper, we demonstrate that HMAC-Whirlpool is vulnerable to power analysis attacks. We designed two types of attacks: one is based on Differential Power Analysis (DPA) and the other on Correlation Power Analysis (CPA). We successfully launched the attacks at HMAC-Whirlpool running on an Atmel AVR processor. We also compared the attacks in terms of the number of power traces needed.**

***Keywords*-HMAC, Whirlpool, DPA, CPA**

## I. Introduction

As a cryptographic primitive, cryptographic hash functions are widely used in many applications such as digital signatures and message authentication codes (MAC). One type of MAC is keyed-Hash Message Authentication Codes (HMAC), which is computed with a cryptographic hash function. A secret key is involved so only the parties who know the key can generate a correct HMAC. Assuming the key is kept securely, the cryptographic strength of the HMAC depends on the cryptographic strength of the underlying hash function.

There has been a lot of work on the design and analysis of cryptographic hash functions, especially after Wang *et al* found efficient collision attacks on a set of hash functions including MD4, MD5, SHA-0 and SHA-1 [9], [10]. Although no collision pairs have been reported for other algorithms in SHA family, including SHA-256, SHA-384, SHA-512, it is believed that they are vulnerable to the same type of attacks because they are designed with similar principles: a Merkle-Damgard model with a compression function consisting of logical operations.

Whirlpool [12], designed by V. Rijmen and P. Barreto, is a cryptographic hash function based on a 512-bits block cipher that is very similar to AES [6]. It was adopted by the International Organization for Standardization (ISO) in the ISO/IEC 10118-3:2004 standard [2], and later approved by NESSIE [3]. In [22], a rebound attack is designed to break a reduced Whirlpool with at most seven rounds. No attack has been found to break the full ten rounds of Whirlpool.

Since Whirlpool is a block cipher based hash function, its performance is not as good as many other functions such as SHA-1 and SHA-2. As a result, it is not widely adopted yet. However, efficient implementations of Whirlpool have been reported recently, in which Whirlpool can run faster than SHA-2 [11].

HMAC can adopt any hash function including Whirlpool. Considering the collision attacks proposed by Wang *et al* [9], [10], Whirlpool seems to be a better candidate if security needs to be emphasized. However, in this paper, we demonstrate that HMAC-Whirlpool is vulnerable to power analysis attacks [13].

Power analysis is a type of side channel attacks that exploit the power consumptions of cryptographic devices to reveal the secret data used in cryptographic computations. Differential Power Analysis (DPA) exploits the relationship between power consumptions and data generated during computation. In a typical DPA attack, adversaries collect a set of power traces and use statistical methods to check whether a specific value is generated. They can deduce the secrets by observing how input data affect the watched value. A more advanced technique is Correlation Power Analysis (CPA) which detects the keys by analyzing the correlation coefficient of the computed data and the real power dissipation.

Prior work has studied DPA on HMAC based on several hash functions. For example, HMAC-SHA-2 has been demonstrated vulnerable to DPA in [14]. Some recent work aimed to launch the power analysis attacks to MACs(HMAC/NMAC) based on the SHA-3 candidates [16]. McEvoy *et al* studied DPA on HMAC with a class of hash functions [15]. Although it is known that HMAC-Whirlpool is potentially vulnerable to DPA, no attack has been reported in details. In this paper, we demonstrate both DPA and CPA on HMAC-Whirlpool. To the best of our knowledge, these are the first successful side-channel attacks on HMAC-Whirlpool in real systems.

The organization of this paper is as follows. Section II briefly introduces HMAC-Whirlpool and power analysis attacks. Section III presents our attacks. Section IV validates our methods with experiments. We conclude the paper in Section V.

## II. Background

### A. Hash Function and Whirlpool

A cryptographic hash function $H$ is a transformation that takes an input $m$ and returns a string $h$ with a fixed length $n$.

The output of the hash function $h$ is called a *hash value* or a *digest*. An $n$-bit hash function outputs a hash value of $n$ bits. Normally the maximal length of $m$ is much larger than $n$. We use $h = H(m)$ to denote that $h$ is a hash value generated by hash function $H$ on input $m$. No secret parameters exist in the hash function. The properties of cryptographic hash functions include one-wayness, second pre-image resistance, and collision-resistance [17].

Most popular hash functions follow the Merkle-Damgard model, which has shown good properties over the years [18], [19]. In this model, the message is padded and divided into blocks of the same length. The blocks are then processed sequentially with a function, usually called a compression function (CF). It can be proven that if the compression function is collision resistant, the hash function constructed from the Merkle-Damgard model is also collision resistant. Typically, a CF $f$ transforms two fixed length inputs to an output of the same size as one of the inputs.

There are three categories of hash functions according to how CF is constructed [17]. The first one is specially designed function. The second one is based on modular arithmetic. The third one is based on block ciphers. The advantage of utilizing block ciphers is that existing implementation of the block cipher can be leveraged to provide the hash functionality with little additional cost.

There are some different constructions to turn a block cipher into a CF, such as Davis-Meyer, Matyas-Meyaer-Oseas, Miyaguchi-Preneel etc [17]. The Miyaguchi-Preneel method starts with an initial hash value and updates it as message blocks are processed. A block of message $m_i$ is fed to the block cipher as plaintext. The intermediate hash value $H_{i-1}$ is used as the key. The output of the block cipher is XORed ($\oplus$) with $m_i$ and $H_{i-1}$ to generate the new intermediate hash value $H_i$. Miyaguchi-Preneel can be described as:

$$H_i = E_{H_{i-1}}(m_i) \oplus H_{i-1} \oplus m_i \tag{1}$$

Whirlpool is a block cipher based hash function built with the Miyaguchi-Preneel method. It takes a message of length less than $2^{256}$ bits and produces a hash value of 512 bits. Given an input message $m$ of bit length $L < 2^{256}$, $m$ is padded so that the message length is $t \times 512$ bits, where $t$ is an integer. The padded message can be partitioned in $t$ blocks of 512 bits, $m_1, m_2, \ldots, m_t$, which are fed into the compression function iteratively as described in the Merkle-Damgard model. The initial hash value $H_0$ is set to a block of 0's. The output of the last iteration $H_t$ is the digest.

The compression function $f$ in Whirlpool is based on a 512-bit internal block cipher $W$, following Miyaguchi-Preneel method. The cipher $W$ is similar to AES [6]. Fig. 1 illustrates the structure of Whirlpool.

Whirlpool can be represented by the following equations.

$$\begin{aligned} \eta_i &= \mu(m_i) \\ H_0 &= \mu(0) \\ H_i &= W[H_{i-1}](\eta_i) \oplus H_{i-1} \oplus \eta_i, 1 \le i \le t \end{aligned} \tag{2}$$

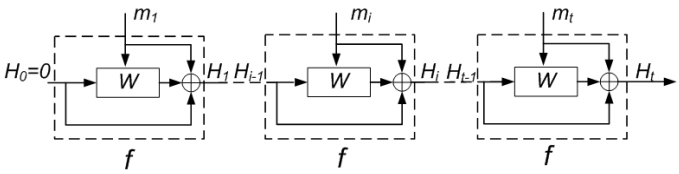



Fig. 1. Diagram of Whirlpool

$\mu$ is a function that formats a string of bits to a matrix-like array.

The block cipher $W$ operates on a 512-bit plaintext block plus a 512-bit key and generates a 512-bit ciphertext. So in this paper, a *block* has 512 bits if not otherwise specified. The *key* used in $W$ is the previous hash value. The plaintext is a padded message block $m_i$, where $1 \le i \le t$.

There are four transformations in both *encryption* and *key scheduling*: substitute bytes(**SB**), shift columns(**SC**), mix rows(**MR**), and add key(**AK**). Here we take the notations in [12]. All four transformations are based on blocks which are represented as $8 \times 8$ matrices of bytes. We briefly talk about **SB** and **AK** transformations in next paragraphs, and skipped **SC** and **MR** because these two are not critical in our attacks.

**SB** is a nonlinear layer based on a lookup table, called SBOX, which is arranged as an $8 \times 8$ matrix of bytes. Given an input byte, SBOX takes the leftmost 4 bits as row index and the rest as column index, and returns a byte value. Considering an input matrix $a$ and output matrix $b$, **SB** can be represented as:

$$\mathbf{SB}(a) = b \Leftrightarrow b_{ij} = \text{SBOX}[a_{ij}], 0 \le i, j \le 7 \tag{3}$$

**AK** uses a bitwise XOR to add the round key $k$:

$$\mathbf{AK}[k](a) = b \Leftrightarrow b_{ij} = a_{ij} \oplus k_{ij}, 0 \le i, j \le 7 \tag{4}$$

The round function **RF** is a combination of the four operations:

$$\mathbf{RF}[k] = \mathbf{AK}[k] \circ \mathbf{MR} \circ \mathbf{SC} \circ \mathbf{SB} \tag{5}$$

The *key scheduling* uses **RF** to generate a sequence of rounds keys $K_0, K_1, \ldots, K_R$:

$$\begin{aligned} K_0 &= K \\ K_r &= \mathbf{RF}[c_r](K_{r-1}), r > 0 \end{aligned} \tag{6}$$

where $K$ is the intermediate hash value, and $c_r$ is the round constant for the $r$-th round [1].

The *encryption* step also uses **RF**. It takes the message block $m_i$ as plaintext and $K_r$ as the key. Finally the internal block cipher $W$ is defined as:

$$W[K] = (\bigcirc_{1}^{r=R} \mathbf{RF}[K_r]) \circ \mathbf{AK}[K_0] \tag{7}$$

Fig. 2 shows one iteration of Whirlpool. The grey box is the block cipher $W$.

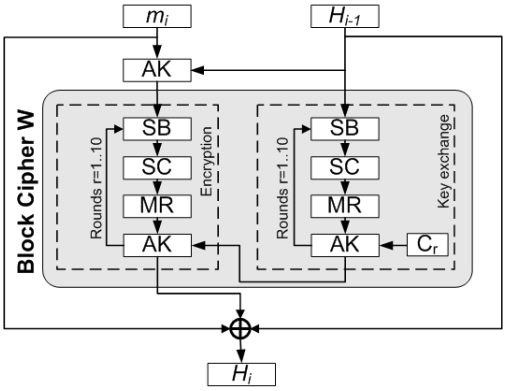



Fig. 2. An iteration of Whirlpool

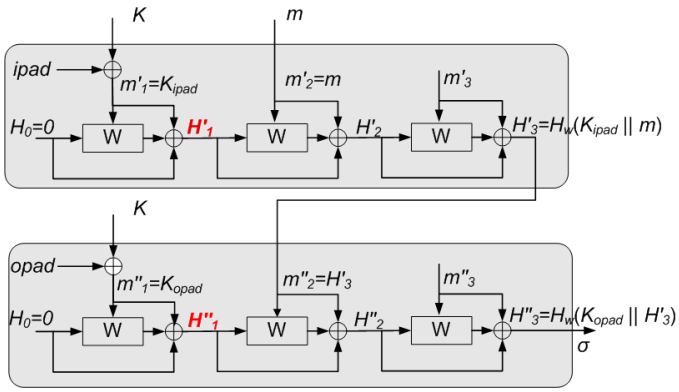



Fig. 3. HMAC-Whirlpool where both $K$ and $m$ are one-block long

### B. HMAC-Whirlpool

The keyed-Hash Message Authentication Code, abbreviated as HMAC, can work with any cryptographic hash function [8]. We will refer to HMAC with Whirlpool as HMAC-Whirlpool or HMAC$_w$.

Let $K$ be the secret key and $m$ a message. HMAC-Whirlpool with $K$ and $m$ can be calculated as follows:

$$\text{HMAC}_w(m, K) = H_w(K_{opad} \parallel H_w(K_{ipad} \parallel m)) \quad (8)$$

where $H_w$ is the Whirlpool hash function, $\parallel$ denotes concatenation, $K_{opad} = K_0 \oplus opad$ and $K_{ipad} = K_0 \oplus ipad$. $K_0$ is a block derived from $K$, either $K$ padded with 0's or $H_w(K)$. The outer padding $opad$ and the inner padding $ipad$ are two one-block long constants.

Fig. 3 illustrates HMAC$_w$ with a 512-bits secret key $K$ and a 512-bits input message $m$. Two grey boxes indicate the two runs of Whirlpool, denoted as $H'_w$ and $H''_w$. $K_{opad}$ and $K_{ipad}$ are sent to $H'_w$ and $H''_w$ respectively. In $H'_w$, three message blocks enter $W$, denoted as $m'_i, i \in [1..3]$ where $m'_1 = K_{ipad}$ and $m'_2 = m$. $m'_3$ is the padding block generated by Whirlpool. The output of the first run of Whirlpool $H'_w = H'_3 = H_w(K_{ipad} \parallel m)$ is sent to the second run of Whirlpool $H''_w$ as the second input block. Since $H'_3$ is also one block long, the padding part $m''_3$ is exactly same as $m'_3$. The final result of HMAC$_w$ is $H''_3 = H_w(K_{opad} \parallel H'_3)$.

### C. Differential Power Analysis

Power analysis is a type of side channel attacks that exploits the power consumptions to reveal the secret key in a device. There are two main types of power analyzes: Simple Power Analysis (SPA) and Differential Power Analysis (DPA) [13]. SPA exploits the correlation between the power outputs and the operations. Comparing with DPA, SPA is easy to launch so is the countermeasure design against SPA. DPA exploits the relationship between power consumptions and data values generated during computation. Adversaries collect a set of power traces and use statistical methods to check whether a specific value is generated during cryptographic computations. They can then deduce the secrets by observing how input data affect the targeted values.

DPA was first introduced by P. Kocher [13] to find the secret key in DES [7]. The adversary randomly generates $N$ different inputs and collects the corresponding power traces. Then he defines a funciton $D$ and guesses the value of some bits in the subkey in round 16. For each guess, he uses $D$ to separate traces into two sets: $S_1$ and $S_0$. All the power traces with $D = 1$ are placed in $S_1$, and others in $S_0$. The adversary calculates the average of each set and compare the average power traces by subtracting one from the other. If the guess is correct, there will be observable differences between the two average power traces, corresponding to the different power consumption when 0 and 1 are produced. If the guess is wrong, the two average power traces are similar. In the attacks on DES, an attacker can guess a 6-bit subkey value and check it with power traces. A correct value can be found with 32 trials. Repeating the process targeting different set of bits reveals more bits in subkeys.

DPA is a very powerful attack. It can be used to break many public-key algorithms. In this paper, we describe how to launch a DPA attack on HMAC-Whirlpool.

### D. Correlation Power Analysis

Different from DPA, Correlation Power Analysis (CPA) deduces the correct key by using correlation coefficient of statistics. CPA is first introduced in [23] on AES.

In power analysis, it can be considered that the power dissipation of an operation at a specific time is proportional to the hamming weight of the processing data. Suppose $W$ is the random variable of the measured power and $H$ is the random variable of the hamming weight of the data $D$. The basic hamming weight model for the data dependency can be described as

$$W = aH(D) + b \quad (9)$$

where $a$ is a scalar and $b$ is the random variable for all the other power consumption of a chip. More accurate power model such as hamming distance model can be used if some reference states are predictable.

The correlation coefficient $\rho_{W,H}$ between $W$ and $H$ with their expected values $\mu_W$ and $\mu_H$, and standard deviations $\sigma_W$ and $\sigma_H$ can be calculated as

$$\rho_{W,H} = \frac{Cov(W,H)}{\sigma_W \sigma_H} = \frac{E((W-\mu_W)(H-\mu_H))}{\sqrt{D(W)}\sqrt{D(H)}} \quad (10)$$

where $E$ is the expect value function and $D$ is the variance function.

The correlation coefficient indicates how two random variables matches each other. In a real CPA attack, the value of a secret key is hypothesized and then the hamming weight of some intermediate value is calculated. The higher absolute value of $\rho_{W,H}$ is, the better correlation matches between the measured power consumption and the hypothetical power consumption (hamming weight). The highest absolute value of $\rho_{W,H}$ suggests the correct hypothesized key.

Compared with DPA, CPA requires less number of power traces to launch a successful attack. This is because that in DPA, all the unpredicted data bits penalized the signal to noise ratio (SNR) [23], [24]. The SNR of DPA could be improved if multiple bits are used in prediction [21].

## III. Attacks to HMAC-Whirlpool

### A. Scenario and Assumption

Suppose a communication channel $\mathcal{V}$ is established between two parties: the sender $\mathcal{S}$ and the receiver $\mathcal{R}$. A secret key $K$ is shared between $\mathcal{S}$ and $\mathcal{R}$. $K$ is stored securely, e.g., in a tamper-resistant hardware $\mathcal{Z}$ that implements HMAC$_w$. Given a message $m$, $\mathcal{S}$ generates the signature $\sigma = \text{HMAC}_w(m, K)$ using $\mathcal{Z}$. He sends both the message $m$ and signature $\sigma$ to $\mathcal{R}$.

After receiving $(m', \sigma')$, $\mathcal{R}$ wants to verify that the message he received $(m')$ is really from $\mathcal{S}$ and has not been changed over $\mathcal{V}$. He can calculate the signature $\sigma''$ using $\mathcal{Z}$ with the pair $(K,m')$. $\mathcal{R}$ compares $\sigma'$ and $\sigma''$. If $\sigma' \neq \sigma''$, $\mathcal{R}$ rejects $m'$. Otherwise, $\mathcal{R}$ accepts $m'$.

Now assume there is an adversary $\mathcal{A}$ not knowing $K$. His goal is to forge $\mathcal{S}$'s signature. He can pick a message $\overline{m}$ and compute $\overline{\sigma}$. He then sends the message-signature pair $(\overline{m}, \overline{\sigma})$ to $\mathcal{R}$. $\mathcal{R}$ *verifies* that $\text{HMAC}_w(\overline{m}, K) = \overline{\sigma}$ and *accepts* $\overline{m}$.

We have the following additional assumptions.

1) $\mathcal{A}$ can use $\mathcal{Z}$ to sign as many messages as $\mathcal{A}$ wants.
2) $\mathcal{A}$ can specify the messages arbitrarily.
3) $\mathcal{A}$ can measure the power consumed by $\mathcal{Z}$ during the signing process.

In this paper we focus on $\mathcal{Z}$ at the sender side. Nevertheless, the attack can also be applied to the device at the receiver side by an attacker asking the device to verify received signatures.

### B. Attack Overview

To attack HMAC$_w$, one can aim to get the secret key $K$. Knowing $K$, $\mathcal{A}$ can easily forge $\mathcal{S}$'s signature. However, it is difficult to find out $K$ because $K$ is kept securely and the hash function has good one-way property.

An alternative is to find the intermediate hash value after $K$ is used. In HMAC$_w$, $K$ only affects $H_1$ (the intermediate hash value after the first block of the message is processed) in both runs of the hash functions. $\mathcal{A}$ can discover the value of $H_1$ with the aid of power analysis. Suppose $\mathcal{S}$ wants to sign $m$, he sends $m$ to $\mathcal{Z}$, which performs HMAC$_w$ as illustrated in Fig. 3. We can observe that both $H'_1$ and $H''_1$ in Fig. 3 do not change if $K$ is fixed. If $H'_1$ and $H''_1$ are known, which depend only on $K$, $\sigma = H''_3$ can be computed easily. Giving an arbitrary message $m$, $\mathcal{A}$ can always generate the same signature as $\mathcal{S}$ would, without knowing the value of $K$.

In summary, the basic idea of our attack is to find out $H'_1$ and $H''_1$ (marked as red in Fig. 3) with power analysis.

### C. Power Analysis Attacks

*1) DPA:* Our strategy is to view $H'_1$ and $H''_1$ as sets of bytes and we try to identify their values one by one. Our attack includes four steps: *messages preparation*, *trace collection*, *data analysis*, and *signature forge*.

**Messages Preparation.** Let IM be a set of messages chosen by $\mathcal{A}$ and $N$ is the total number of messages. We use IM$[i]$ to denote message $i$, where $0 \leq i \leq N$. For simplicity, each message has only one block, i.e., 64 bytes. We use IM$[i][j]$ to denote byte $j$ in IM$[i]$, where $0 \leq j \leq 63$. IM$[m..n][j]$ denotes byte $j$ from messages $m, m+1, \ldots, n$.

Since $\mathcal{A}$ can choose any messages he wants, the messages may be generated randomly. However, to make sure the distribution of the value of bytes with the same $j$ is uniform even for a small number of messages, $\mathcal{A}$ can use permutations. He can construct IM as follows: IM$[0..255][j] =$ PERMUTE$(0, 1, 2, \ldots, 255)$ for $0 \leq j \leq 63$, where PERMUTE is a function that permutes its input randomly. In this way, IM$[0..255][j]$ covers all possible values for byte $j$, $0 \leq j \leq 63$. The process can be repeated until $N$ messages are generated.

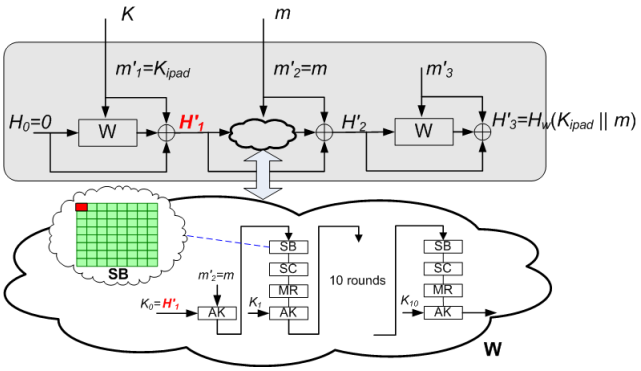



Fig. 4. An illustration of the attack details

**Trace Collection.** $\mathcal{A}$ feeds the prepared messages to $\mathcal{Z}$ and measures the power consumption of $\mathcal{Z}$ when the messages are being signed. So $\mathcal{A}$ has $N$ power traces. Suppose each trace has $T$ sampling points. We use P$[k]$ to denote power trace $k$ and P$[k][t]$ to denote the sampling point at time $t$ in power trace $k$, where $0 \leq k < N$ and $0 \leq t < T$.

**Data Analysis.** After the power traces are collected, $\mathcal{A}$ analyzes the traces and finds out the value of $H_1'$ and $H_1''$. Here, we take the first byte of $H_1'$, denoted as $H_1'[0]$, as an example to illustrate the data analysis process. Note that similar method can also be applied to $H_1''$.

When signing a message $m$, $\mathcal{Z}$ generates $H_1'$ from $K$ first. Then, it invokes $W$ again, with $H_1'$ as the key and the prepared message $m$ as the plaintext. Fig. 4 shows the process. We target $W$ in this iteration. The targeted operation is the table lookup that implements the **SB** transformation. In the first **SB** transformation, marked as red in Fig. 4, the index of the table lookup operation is the XOR of $H_1'[0]$ and $m[0]$, i.e., the first byte of $H_1'$ and $m$. When collecting power traces, $m$ is IM$[i]$, where $0 \leq i < N$.

Targeting this operation, we apply DPA attacks. We guess the value of $H_1'[0]$ is $g$. We define a selection function $D$ which is the least significant bit of the output of the table lookup operation.

$$D(k) = 1 \quad \& \quad \text{SBOX}[g \oplus \text{IM}[k][0]] \tag{11}$$

We separate $N$ power traces into two sets: $S_1$ and $S_0$. If $D(k) = 1$, P$[k]$ will be put into $S_1$, otherwise $S_0$.

$$\begin{aligned} S_0 &= \{\text{P}[k] | D(k) = 0, 0 \leq k < N\} \\ S_1 &= \{\text{P}[k] | D(k) = 1, 0 \leq k < N\} \end{aligned} \tag{12}$$

Next, we compute the average power for each set.

$$\begin{aligned} \text{P}_0[i] &= \frac{1}{|S_0|} \sum_{\text{P}[k] \in S_0} \text{P}[k][i], 0 \leq i < T \\ \text{P}_1[i] &= \frac{1}{|S_1|} \sum_{\text{P}[k] \in S_1} \text{P}[k][i], 0 \leq i < T \end{aligned} \tag{13}$$

Then we compute the difference of $\text{P}_0$ and $\text{P}_1$. The maximal difference is considered as the rank of guess $g$.

$$\text{Rank}(g) = \max_{0 \leq i < T} (\text{P}_1[i] - \text{P}_0[i]) \tag{14}$$

We compute the rank for all possible values (0 to 255). The value with the highest rank is the correct guess of $H_1'[0]$.

$$H_1'[0] = \{v \mid \text{Rank}(v) = \max_{0 \leq g < 255} \text{Rank}(g)\} \tag{15}$$

Repeating the process, all the bytes in $H_1'$ can be revealed. When $H_1'$ is known, we can apply similar methods to find out $H_1''$.

**Signature Forge.** With DPA, $\mathcal{A}$ can find out the value of two intermediates in HMAC$_w$, $H_1'$ and $H_1''$. Then, $\mathcal{A}$ can forge any pair of $(\overline{m}, \overline{\sigma})$ by the following steps.

1) $\mathcal{A}$ picks any message $\overline{m}$ that he wants to send to $\mathcal{R}$.
2) He feeds $\overline{m}$ to a modified Whirlpool hash function with the initial hash value $H_0$ set to $H_1'$. The result is denoted as $\overline{H_3'}$.
3) He feeds $\overline{H_3'}$ to another modified Whirlpool hash function with the initial hash value $H_0$ set to $H_1''$. The result is denoted as $\overline{H_3''}$.
4) He sends the pair $(\overline{m}, \overline{\sigma})$ to $\mathcal{R}$ where $\overline{\sigma} = \overline{H_3''}$.

*2) CPA:* The flow of CPA attack is similar to that of DPA in III-C1. The only difference is the data analysis part. In the following, we use $H_1'[0]$ as an example to illustrate the attack.

For each guessed value $g$ of $H_1'[0]$, we take the first byte of input messages IM$[\cdot][0]$ and calculate the hamming weight of the output of SBOX. Thus we get one data set HW$[\cdot][0]$ with $N$ values, which has all the predicted hamming weight. Note $N$ is the number of messages. Then we calculate the correlation coefficient of HW$[\cdot][0]$ and the actual measured power traces at each sampling point P$[\cdot][t]$. As a result, we obtain a correlation coefficient trace C$[g][\cdot]$ which is corresponding to each guess. Since there are only 256 possible values of $g$, we can therefore have 256 correlation coefficient traces. Among all 256 traces, we can distinguish one from all others. In particular, at some time points, the absolute value of one coefficient trace, where the guess is correct, is much larger than all other traces.

To avoid ghost spikes [23], we do not use the coefficients to rank guesses directly. Instead, we use the difference between the top two of correlation coefficients. Suppose at time $t$, the top two coefficients are C$[g_1][t]$ and C$[g_2][t]$. The value we used for ranking guesses is calculated as

$$\begin{aligned} \text{Rank}[t].v &= \text{C}[g_1][t] - \text{C}[g_2][t] \\ \text{Rank}[t].k &= g_1 \end{aligned} \tag{16}$$

Let the total number of sampling points in a power trace is $T$. We have $T$ rank values. The guess associated with the highest rank is the correct guess of $H_1'[0]$.

$$H_1'[0] = \{\text{Rank}[t_{max}].k \mid \text{Rank}[t_{max}].v = \max_{0 \leq t < T} \text{Rank}[t].v\} \tag{17}$$

## IV. Experiment

### A. Environment Setting

We implemented HMAC-Whirlpool on ATMEGA324P from Atmel Inc. and launched both DPA and CPA attacks successfully.

ATMEGA324P is an AVR® 8-bit microcontroller with 1KB EEPROM, 2KB SRAM and 32KB Flash. It can run at most 20 MHz clock rate. We implement Whirlpool in C language and program the flash on ATMEGA324P. The SBOX is implemented as a lookup table which takes one byte as the index and returns one byte as the output.

A diagram of the environment setting is shown in Fig.5. We put a small resistor R in serial with ATMEGA324P. The power supply, 6624A from Agilent Inc., has a constant output voltage. The voltage drop on R reflects the power consumption of ATMEGA324P since the currents through R and ATMEGA324P are the same.

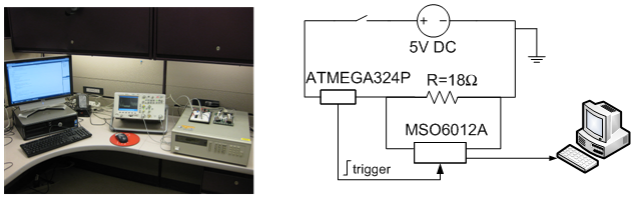



Fig. 5. The experiment setting of power analysis attacks on HMAC-Whirlpool

We use MSO6012A, a digital oscilloscope, to measure the voltage drop on the resistor. MSO6012A has a maximal sampling rate as 2GSa/s and the minimal voltage resolution it can measure is 0.3125mV. The oscilloscope is connected to a PC via a USB interface. The sampled data are transferred to the PC. In our experiment, we set $V_{cc} = 5$V. The microcontroller runs at 8MHz. The oscilloscope samples 100M points per second. The resistance of R is 18.2Ω.

### B. Experiment Result

In our attacks, we generate 256 messages with permutation. We collect $N$ power traces by feeding each message to ATMEGA324P many times. We want to check the attacking effort needed for successful DPA and CPA attacks. The effectiveness of the attacks is compared in terms of the number of power traces needed to reveal all 64 bytes of $H'_1$. Similar result can also be achieved for $H''_1$.

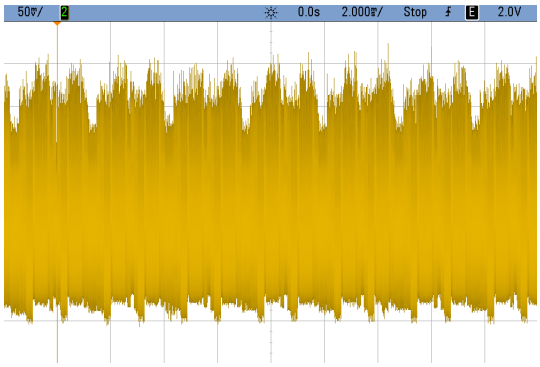



Fig. 6. A real power trace of HMAC-Whirlpool on ATMEGA324P

Fig. 6 shows the real power trace of one measurement of HMAC-Whirlpool running on ATMEGA324P. The total time duration showed on the scope screen covered the calculation of the 64 SBOX table lookup operations.

Fig. 7(a) and (b) show the ranks for all possible values of $H'_1[0]$ and $H'_1[1]$ in DPA attacks. $N = 2048$. The correct value of $H'_1[0]$ is 141 and that of $H'_1[1]$ is 169. In the figures, we can clearly see that the correct guess has the highest rank for both $H'_1[0]$ and $H'_1[1]$, much higher than the ranks of other values. Increasing $N$ can increase the SNR (Signal to Noise Ratio) of power traces and make the result more likely to be correct. Another way to improve the accuracy is to adopt the multi-bit DPA [21], which we will explore in future.

Fig. 8(a) and (b) show the coefficient curves for different guesses in CPA attacks. The curve corresponding to the correct guess is drawn in red, other curves associative with wrong guesses are in green. The spike of the correct guess is easy to observe. Here we use $N = 2048$ power traces.

Both DPA and CPA can reveal the secret in HMAC-Whirlpool. The question is which attack is more powerful. We can compare the two attacks by counting the number of power traces needed to detect all the 64 bytes of $H'_1$. The less power traces required, the more effective an attack is. We can also compare the attacks by the number of bytes revealed with the same set of power traces.

TABLE I
NUMBER OF POWER TRACES USED IN DPA AND CPA

| $N$ | $DPA$ | $DPA^*$ | $CPA$ |
|---|---|---|---|
| 256 | 0 | 1 | 22 |
| 512 | 0 | 8 | 38 |
| 768 | 5 | 17 | 55 |
| 1024 | 5 | 22 | 62 |
| 2048 | 25 | 45 | 64 |
| 3072 | 34 | 62 | 64 |
| 4096 | 38 | 61 | 64 |
| 5120 | 40 | 64 | 64 |
| 6144 | 49 | 64 | 64 |
| 7168 | 50 | 64 | 64 |
| 8192 | 50 | 64 | 64 |

Table I lists the number of detected bytes of $H'_1$ with different numbers of power traces. The second column is the regular DPA. The third column, named as DPA′, is a variant of DPA where we search the DPA spike in a short time range where the targeted operation is performed. This additional time information narrows down the search range and improves DPA. The fourth column lists the result of CPA. As we can see from Table I, CPA uses only 2048 power traces to reveal all 64 bytes while DPA′ needs almost 2.5 times power traces. A regular DPA may need more than 8096 power traces if the detection of all 64 bytes of $H'_1$ is required. Our results show that CPA is more effective than DPA on HMAC-Whirlpool.

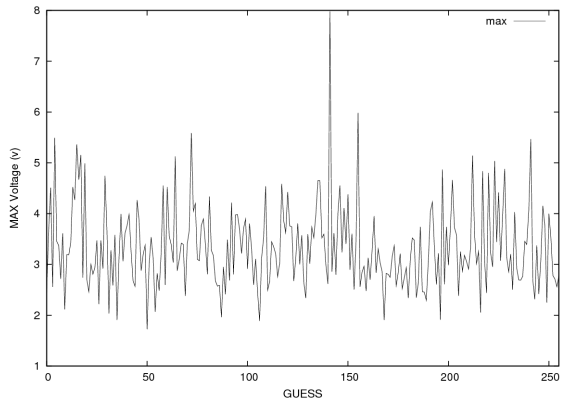
## V. CONCLUSIONS AND FUTURE WORK

In this paper, we propose two power analysis attacks on HMAC-Whirlpool. Targeting at the table lookup operation in Whirlpool, we can reveal two intermediate values with both DPA and CPA attacks. As a result, a valid signature can be forged without knowing the key. We have successfully launched attacks on HMAC-Whirlpool running on an 8-bit Atmel processor.

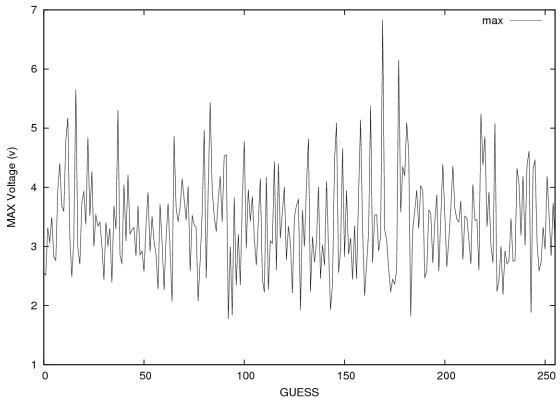
In the future, we will try to break HMAC-Whirlpool on other platforms and experiment with multi-bit DPA. At the same time, we will explore the countermeasures, especially software countermeasures that have reasonable overhead and can be deployed in existing systems.

## REFERENCES

[1] The WHIRLPOOL Hash Function. *http://www.larc.usp.br/ pbarreto/WhirlpoolPage.html*

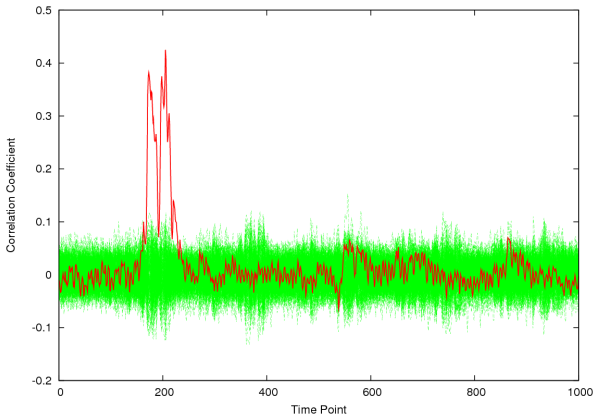



(a) DPA attack on $H_1'[0]$ (141 is the correct value).

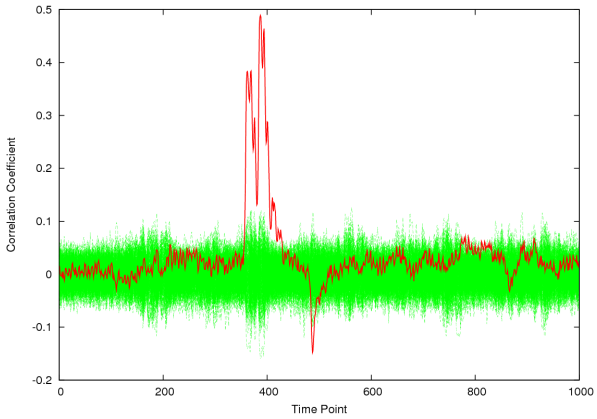



(b) DPA attack on $H_1'[1]$ (169 is the correct value).

Fig. 7. Ranks of all possible values of $H_1'[0]$ and $H_1'[1]$ in DPA, $N = 2048$.




(a) Correlation coefficient curve revealing $H_1'[0]$




(b) Correlation coefficient curve revealing $H_1'[1]$

Fig. 8. Correlation coefficient curves for $H_1'[0]$ and $H_1'[1]$ in CPA, $N = 2048$.


[2] ISO/IEC 10118-3:2004. *http://www.iso.org/*

[3] NESSIE. New European Sechemes for Signatures, Integrity, and Encryption. IST-1999-12324. *http://cryptonessie.org/*

[4] NIST Cryptographic Hash Algorithm Competition. *http://csrc.nist.gov/groups/ST/hash/sha-3/index.html*

[5] J. Daemen, and V. Rijmen. The Wide Trail Design Strategy In *LNCS*, vol. 2260, pp.222-238, January 2001.

[6] Advanced Encryption Standard. NIST In *U.S. FIPS PUB 197*, 2001.

[7] Data Encryption Standard. NIST In *U.S. FIPS PUB 46-3*, 1999.

[8] The Keyed-Hash Message Authentication Code (HMAC). NIST In *U.S. FIPS PUB 198*, October 2002.

[9] X. Wang, X. Lai, D. Feng, H. Chen, and X. Yu. Cryptanalysis of the Hash Functions MD4 and RIPEMD. In *LNCS*, vol. 3494, pp.1-18, May 2005.

[10] X. Wang, Y. Yin, and H. Yu. Finding Collisions in the Full SHA-1. In *LNCS*, vol. 3621, pp.17-36, August 2005.

[11] Y. Hilewitz, Y. Yin, and R. Lee. Accelerating the Whirlpool Hash Function Using Parallel Table Lookup and Fast Cyclical Permutation. In *LNCS, Fast Software Encryption*, vol. 5086, pp.173-188, July 2008.

[12] W. Stallings. The Whirlpool Secure Hash Function. In *Cryptologia*, vol. 30, issue. 1, pp.55-67, January 2006.

[13] P. Kocher, J. Jaffe, and B. Jun. Differential Power Analysis. In *Proceedings of CRYPTO'99*, pp.388-397, August 1999.

[14] R. McEvoy, M. Tunstall, C. Murphy, and W. Marnane. Differential Power Analysis of HMAC Based on SHA-2, and Countermeasures. In *WISA 2007, LNCS 4867*, pp.317-332, September 2007.

[15] R. McEvoy, M. Tunstall, C. Whelan, N. Hanley C. Murphy, and W. Marnane. Differential Power Analysis of HMAC Algorithm. poster paper In *CHES 2007*, September 2007.

[16] P. Gauravaram, and K. Okeya. Side Channel Analysis of Some Hash Based MACs: A Response to SHA-3 Requirements. poster paper In *ICICS 2008, LNCS 5308*, pp.111-127, 2008.

[17] A. Menezes, P. Oorschot, and S. Vanstone. Handbook of Applied Cryptography. *http://www.cacr.math.uwaterloo.ca/hac/*.

[18] I. Damgard. A Design Principle for Hash Functions. In *Gilles Brassard, editor, CRYPTO, LNCS 435*, pp.416-427, 1989.

[19] R. Merkle. One Way Hash Functions and DES. In *Gilles Brassard, editor, CRYPTO, LNCS 435*, pp.428-446, 1989.

[20] F. MacWilliams, and N. Sloane. The Theory of Error-Correcting Codes. In *North-Holland Mathematical Library*, vol. 16, 1977.

[21] T. S. Messerges, E. A. Dabbish, and R. H. Sloan. Examining smart-card security under the threat of power analysis attacks. In *IEEE Transactions on Computers*, vol. 51, no. 5, pp. 541-552, 2002.

[22] F. Mendel, C. Rechberger, M. Schlaffer, and S. Thomsen. The Rebound Attack: Cryptanalysis of Reduced Whirlpool and Grostl. In *FSE 2009*, vol. 51, no. 5, pp. 541-552, May 2009.

[23] E. Brier, C. Clavier, and F. Olivier. Correlation Power Analysis with a Leakage Model. In *CHES 2004*, LNCS, vol. 3156/2004, pp. 135-152, 2004.

[24] E. Brier, C. Clavier, and F. Olivier. Optimal statistical power analysis. *http://eprint.iacr.org/2003/152*.